

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

The Missing Lever in Machine Learning Deployment: Threshold Tuning using Regression Discontinuity

Lucas Squarize Chagas[1], and **Avik Basu**[1,2]

[1]Intuit, Inc., [2]PyOpenSci

Abstract

When deploying machine learning models into production, a critical but often overlooked lever is the decision threshold that converts predictions into actions. In practice, thresholds are typically chosen arbitrarily and left unchanged, leaving substantial value on the table despite heavy investment in model development.

We propose a systematic framework for optimizing thresholds using Regression Discontinuity Design (RDD), an econometric method that exploits the natural experiments created by decision cutoffs. We operationalize this approach into a Python package that automates bandwidth selection, local linear estimation, robustness checks, visualization, and optimization routines. Validated on simulated data and applied within a large-scale production system at Intuit, RDD-based threshold tuning outperformed the alternative methods we tested, improving target metrics without harming guardrails and enabling principled trade-off analysis between competing business objectives. We also delineate where the approach applies: the estimate is local to the cutoff, its precision depends on data density near the threshold, and the tuned value is conditional on the model that produces the score.

Keywords regression discontinuity, threshold tuning, machine learning, causal inference, optimization

1. Motivation

When deploying machine learning models into production, one critical lever is routinely overlooked: the threshold that converts predictions into actions. A recommender system must decide which items to display, a churn model who to target, and a fraud model when to trigger an alert. These thresholds often matter as much for outcomes as the underlying model itself. Yet in practice, they are usually chosen arbitrarily: 50% relevance to show a recommendation, 70% fraud risk to flag a case as fraud, 90% churn probability to trigger a discount. Once set, they are often left unchanged throughout the model's lifecycle.

As a result, firms invest heavily in model development but leave major value untapped by relying on static, guesswork thresholds. We propose a systematic way to optimize thresholds using Regression Discontinuity Design (RDD) [1], an econometric method that leverages arbitrary policy or decision cutoffs to uncover causal relationships.

2. Methodology: Regression Discontinuity to Tune Thresholds

Thresholds create natural experiments: units just above and just below a cutoff are nearly identical, except that one receives a treatment (e.g., a recommendation or a discount) and the other does not. RDD exploits this discontinuity to estimate the causal effect of the treatment for units that are close to the threshold (Figure 1 displays an example).

Published Jul 13, 2026

Correspondence to
Lucas Squarize Chagas
lsquarize@gmail.com

Open Access

Copyright © 2026 Chagas & Basu. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

This turns an arbitrary cutoff into a measurable and optimizable decision rule. By focusing on outcomes near the threshold, RDD can both diagnose whether the current cutoff is adding or destroying value, and optimize by recommending whether the threshold should be raised, lowered, or kept constant to maximize outcomes while preserving guardrails.

We operationalized this approach into a practical methodology that combines causal inference with optimization. Building on recent econometrics research [2], we use fitted values from RDD models to find alternative values for the thresholds in order to maximize business outcomes, with or without guardrail constraints. For example, in Figure 1, the positive impact of a recommendation in the main outcome metric suggests that a reduction in the threshold would generate additional gain.

To scale adoption, we developed a Python package that automates RDD-based threshold optimization. The package includes bandwidth selection, local linear estimation, robustness checks, visualization tools, and optimization routines that identify the best threshold for different scenarios. This enables seamless integration into production ML pipelines without requiring new randomized experiments or uplift modeling. With online iterations, we guarantee that our decision thresholds are always at an optimized level. We recommend the RDD-based threshold tuning as a crucial workflow for ML model deployment (Figure 2).

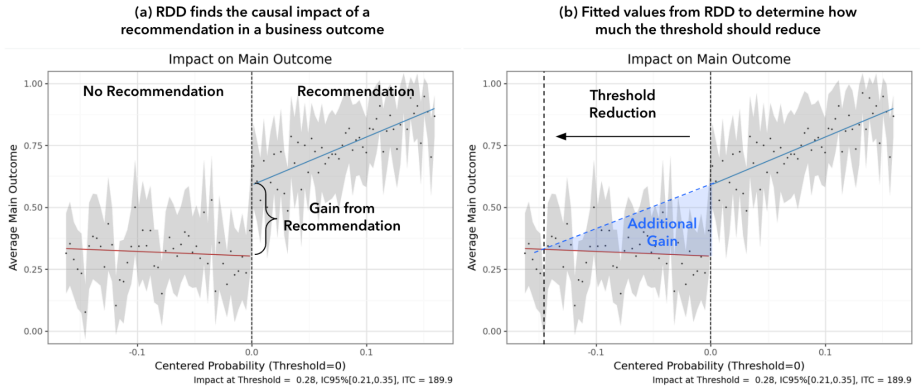


Figure 1. *RDD-based threshold tuning. In the graphs, the estimated score from an ML model is on the horizontal axis, and the average outcome (e.g., conversion) is on the vertical axis. The graphs show the fitted local regressions to the left (red) and right (blue) of a threshold for a recommendation (grey shades are confidence intervals).* ***a)*** *RDD analysis estimates near the threshold show that the recommendation produces a gain in the main outcome.* ***b)*** *Using the fitted values we recommend optimized thresholds aimed at maximizing the business outcome. For illustration, the figures were constructed using simulated data.*

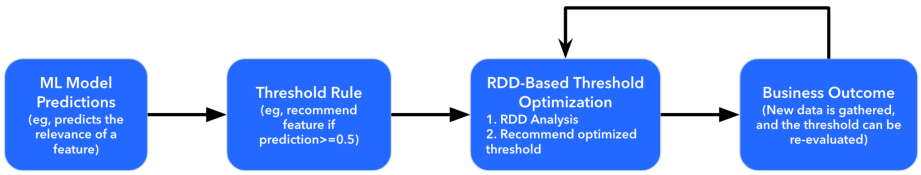


Figure 2. *Workflow for RDD-based threshold tuning. Predictions from an ML model are translated into actions using a threshold. RDD analysis estimates causal effects near the threshold and recommends optimized thresholds that improve business outcomes, considering both the main outcome and possible guardrails.*

3. Applications and Impact

First, we validated the method on simulated data and then applied it within a large-scale system at Intuit. The deployments reported below cover two domains, sequential

recommendation and premium-feature monetization; we discuss in the following section what does and does not carry over to settings with different objectives and constraints. For instance, in a sequential recommendation setting, we originally defined thresholds at 50% for topic recommendations, which still left potential gains on the table. We then tested different approaches to threshold tuning in a real experiment. In one recipe, thresholds were optimized using our RDD-based methodology, while others relied on alternative methods such as arbitrary cutoffs or precision and recall optimization. Results showed that thresholds optimized with RDD performed better than these alternatives, improving our target metrics without harming guardrail metrics.

Another application involved a premium feature recommendation with a trade-off between revenue and units. This is similar to a discount policy that offers a price discount to customers with a high probability of churning, increasing their probability of conversion but reducing the expected revenue per customer. We used our RDD-based method to produce trade-off curves between revenue and conversion, allowing leaders to steer outcomes more scientifically depending on their business goals (Figure 3).

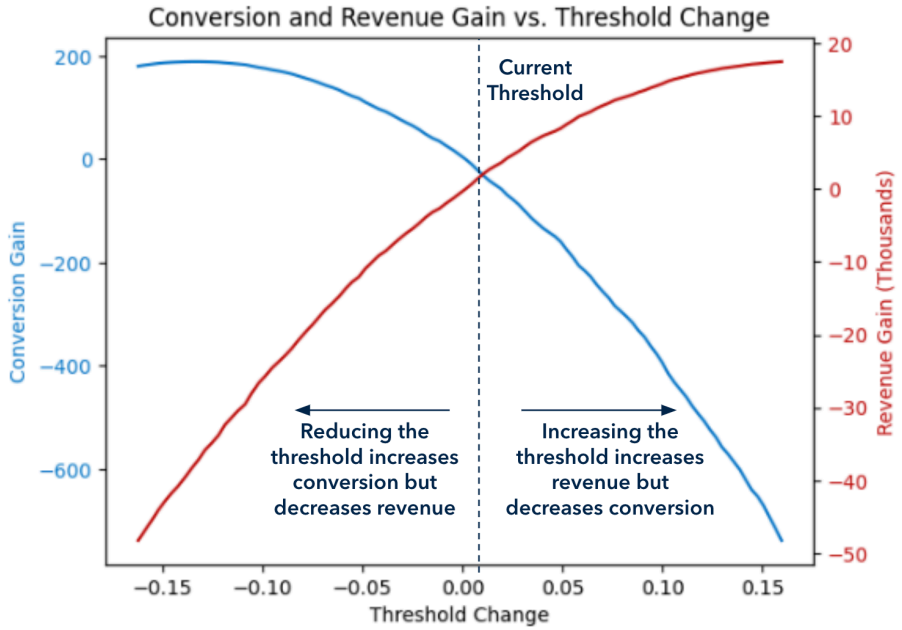


Figure 3. *Business outcome as a function of threshold. RDD-based threshold tuning provides the trade-off between two or more conflicting metrics: the main outcome (e.g., conversion) and a guardrail (e.g., revenue). In this case, a reduction in the threshold increases conversion but reduces revenue, while an increase in the threshold increases revenue but reduces conversion. These curves present leaders the trade-off associated with their business decision regarding the threshold level. For illustration, the figure was constructed using simulated data.*

Across simulated and real-world deployments, the approach has led to measurable improvements in engagement, conversions, and revenue, while preserving quality guardrails. Importantly, it provides a lightweight, repeatable way to adapt thresholds as customer behavior and business priorities evolve.

4. Limitations and Scope

RDD identifies a *local* average treatment effect: the estimate is credible in a neighborhood of the cutoff and says little about units far from it. A recommendation to move the threshold substantially therefore extrapolates beyond the region the design identifies, and the size

of a defensible move is bounded by the bandwidth used for estimation [3]. This is why we frame tuning as an iterative loop (Figure 2) rather than a one-shot optimization: move the threshold modestly, re-estimate on the data generated under the new cutoff, and repeat. The design also rests on assumptions that must be checked rather than assumed. Potential outcomes must be continuous at the cutoff, so that any jump is attributable to the decision rule and not to something else changing at the same score. Units must not be able to precisely manipulate their position relative to the threshold, an assumption that deserves particular scrutiny in adversarial settings where the score is a target to be gamed; density tests around the cutoff [4] are a necessary diagnostic, and our package reports them alongside the effect estimate.

Statistical precision depends on the mass of observations inside the bandwidth, not on the total size of the dataset. Under severe class imbalance, or whenever the score distribution is thin near the cutoff, confidence intervals widen and the recommended threshold becomes correspondingly unstable. We therefore report the effective sample size within the bandwidth and treat a wide interval as evidence for leaving the threshold unchanged, rather than as a point recommendation to act on. A related caution concerns what is being optimized: the business outcomes we target, such as conversion or revenue, are not the same objects as precision and recall, and for rare-event problems the two can move in different directions. A threshold change that improves aggregate business value may leave classification metrics flat or slightly worse, and practitioners with rare-event objectives should specify the outcome they actually care about rather than assuming the two coincide. The difference is one of estimand, not only of units. Precision and recall at a candidate cutoff are computed over the entire score distribution, so they are dominated by observations far from the margin, which are exactly the units a threshold move does not reassign; they summarize how well the score separates labels, not what acting at the margin causes. Restricting attention to a neighborhood of the cutoff is what licenses a causal reading, and it is what the locality noted above buys in exchange for the loss of external validity. The two also sit at different points in the lifecycle: precision and recall are natural during model development, when the score itself is still being selected, whereas the tuning we describe is a post-deployment adjustment to a fixed pipeline, estimated on outcomes that pipeline has already generated.

Our validation covers simulated data together with recommendation and premium-feature monetization settings at Intuit. We do not claim validation in fraud detection, healthcare, credit risk, or other safety-critical domains. Those settings raise concerns the present evaluation does not speak to: scores may be actively manipulated, the costs of false positives and false negatives are sharply asymmetric, and regulatory constraints may make some thresholds inadmissible regardless of measured lift. The framework also inherits the properties of the model that produces the running variable. Because the score itself is the axis along which the discontinuity is measured, a tuned threshold is conditional on a fixed model pipeline; retraining, recalibration, or a shift in feature quality changes the score distribution and requires re-tuning. Reported lift should accordingly be read as the effect of the threshold given that pipeline, not as a property of the threshold alone.

Finally, outcomes and guardrails are measured over a fixed post-decision window. Longer-horizon effects on retention, trust, and future engagement are not captured, and a threshold that improves short-run conversion could in principle degrade them. Nothing in the design forbids a longer window: RDD identifies a discontinuity in whatever outcome is measured against the running variable, and a twelve-month retention outcome is as admissible as a seven-day conversion one. The obstacle is operational rather than inferential. A long-horizon outcome must accumulate under a fixed cutoff before it can be estimated, which bounds how quickly the iterative loop described above can turn and strains the requirement that the scoring pipeline stay unchanged across the measurement window. Substituting short-

term leading indicators is the usual remedy, but a proxy justifies the substitution only if the treatment affects the long-run outcome exclusively through it [5], a condition rarely met by any single indicator and one that constructions such as the surrogate index [6] are designed to relax. We take no position on which surrogates are defensible in the settings we study; treating that question properly is beyond the scope of this paper, and we flag it as the principal obstacle to long-horizon threshold tuning.

RDD is also not the only route to a tuned threshold, and we position it as complementary rather than dominant. Uplift and metalearner approaches estimate heterogeneous effects across the entire score range [7], which RDD does not, but they require randomized or credibly quasi-random assignment. Bayesian optimization [8] and reinforcement learning policies can search the threshold space directly, but need online interaction and many evaluations to converge. What distinguishes RDD is that it requires neither new experiments nor online exploration: it reuses the natural experiment the existing cutoff already creates, which is what makes it deployable against a system already in production. A controlled benchmark against these alternatives remains future work.

5. Contributions and Implications

The examples in the previous section illustrate how the framework translates statistical insights into practical levers for decision-making, without requiring new experiments. Our contributions are fourfold:

- **Conceptual**: Elevating threshold decisions as critical, often-overlooked levers in machine learning deployment.
- **Methodological**: Showing how RDD can be repurposed from evaluation to optimization.
- **Practical**: Delivering a package and workflow that make RDD tuning accessible and scalable.
- **Empirical**: Demonstrating impact in production deployments without costly new experiments.

More broadly, this work positions threshold tuning as a causal optimization problem. By embedding RDD into production pipelines, firms can move from arbitrary cutoffs to adaptive, principled business rules that maximize value at scale.

References

[1] M. D. Cattaneo, N. Idrobo, and R. Titiunik, *A Practical Introduction to Regression Discontinuity Designs*. Cambridge University Press, 2020.

[2] I. Marinescu, S. Triantafillou, and K. Kording, "Regression discontinuity threshold optimization," *PLoS ONE*, vol. 17, no. 11, p. e276755, 2022, doi: 10.1371/journal.pone.0276755.

[3] S. Calonico, M. D. Cattaneo, and R. Titiunik, "Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs," *Econometrica*, vol. 82, no. 6, pp. 2295–2326, 2014, doi: 10.3982/ECTA11757.

[4] J. McCrary, "Manipulation of the running variable in the regression discontinuity design: A density test," *Journal of Econometrics*, vol. 142, no. 2, pp. 698–714, 2008, doi: 10.1016/j.jeconom.2007.05.005.

[5] R. L. Prentice, "Surrogate endpoints in clinical trials: Definition and operational criteria," *Statistics in Medicine*, vol. 8, no. 4, pp. 431–440, 1989, doi: 10.1002/sim.4780080407.

[6] S. Athey, R. Chetty, G. Imbens, and H. Kang, "The Surrogate Index: Combining Short-Term Proxies to Estimate Long-Term Treatment Effects More Rapidly and Precisely," NBER Working Paper 26463, 2019. doi: 10.3386/w26463.

[7] S. R. Künzel, J. S. Sekhon, P. J. Bickel, and B. Yu, "Metalearners for estimating heterogeneous treatment effects using machine learning," *Proceedings of the National Academy of Sciences*, vol. 116, no. 10, pp. 4156–4165, 2019, doi: 10.1073/pnas.1804597116.

[8] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016, doi: 10.1109/JPROC.2015.2494218.